Algorithmique des Tableaux : Parcours complet conditionnel et Modèles de Recherche

Thème principal : Structures de données linéaires et Algorithmes de parcours exhaustif et conditionnel

Synthèse exécutive : Ce cours formalise les mécanismes fondamentaux de manipulation des tableaux à une dimension en algorithmique. Nous y abordons le parcours complet avec traitement conditionnel, la gestion stricte des bornes d'indices (de 0 à N-1) pour éviter les erreurs Out of Bounds, ainsi que les subtilités liées aux éléments adjacents (T[i-1]). Le cours distingue également le parcours exhaustif de la recherche d'une première occurrence (arrêt prématuré vs parcours total).

1. Le Parcours Complet avec Traitement Conditionnel

En informatique, un tableau à une dimension de N éléments est structuré de façon séquentielle en mémoire avec un accès direct par indice. Lorsque le problème exige d'évaluer chaque case sans condition de sortie anticipée (par exemple pour filtrer, compter ou afficher certains éléments particuliers), on utilise un 'parcours complet'. L'utilisation de la structure itérative bornée 'Pour i allant de 0 à N - 1' est prescrite car le nombre d'itérations est fixe et connu d'avance. À l'intérieur de cette boucle, un filtre sélectif 'Si condition(T[i]) Alors' isole les éléments répondant aux critères requis. La vigilance sur les bornes est capitale : tenter d'accéder à l'indice N provoque un dépassement de mémoire (Out of Bounds / Segmentation Fault).

- Parcours exhaustif : inspection systématique de chaque case de l'indice 0 à N-1.
- Traitement conditionné : filtrage des éléments par une structure de décision 'Si... Alors' avant d'appliquer une action.
- Gestion rigoureuse des bornes : en indexation 0, la dernière case valide est à l'indice N-1.

Cas concret enseignant : *C'est exactement comme un vigile à l'entrée d'un club qui contrôle les cartes d'identité de toute la file d'attente, une par une de la première à la dernière personne. Il ne s'arrête pas après avoir laissé entrer un client (ce n'est pas une recherche), il passe en revue l'intégralité de la file et applique un filtre 'Si l'âge >= 18 alors entrer'.*

2. Dépendances entre Élément Courant et Élément Précédent (T[i] et T[i-1])

Lorsqu'un algorithme requiert une comparaison relative entre éléments adjacents (par exemple vérifier si un tableau est trié en comparant T[i] à T[i-1]), l'accès à T[i-1] pour i = 0 engendrerait une tentative d'évaluation de T[-1], ce qui constitue une erreur fatale d'exécution. Pour garantir la sécurité mémoire tout en préservant l'exhaustivité des comparaisons entre paires, la borne inférieure du parcours doit obligatoirement être ajustée à 1. La boucle s'exécutera ainsi de i = 1 jusqu'à N - 1, comparant ainsi correctement chaque élément T[i] à son prédécesseur direct T[i-1].

- Décalage d'indices : comparaison de la case courante i avec la case précédente i-1.
- Ajustement de la borne initiale : démarrer la boucle à i = 1 au lieu de i = 0 pour éviter un indice négatif invalide (T[-1]).

Cas concret enseignant : *Si vous comparez chaque marche d'un escalier avec la marche précédente pour vérifier si l'escalier monte toujours, vous ne pouvez pas commencer votre comparaison sur la première marche (marche 0) car il n'y a pas de marche en dessous. Vous devez obligatoirement vous placer sur la deuxième marche (indice 1) et la comparer à la première (indice 0).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Comment fait-on si la condition dans un parcours complet conditionnel dépend de l'élément précédent (par exemple T[i] > T[i-1]) ?
Réponse du professeur : Pour éviter l'erreur d'indice négatif T[-1] à la première itération, il faut absolument ajuster la borne de départ de la boucle. Au lieu d'écrire 'Pour i allant de 0 à N-1', on écrit 'Pour i allant de 1 à N-1'. Ainsi, lors de la première itération avec i=1, l'algorithme évalue T[1] > T[0], ce qui est parfaitement valide et sécurisé.

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur donnée (par exemple trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il ne faut pas utiliser une 'recherche de première occurrence' (qui s'arrête dès qu'elle trouve le premier 5), mais un 'parcours complet conditionnel'. Vous devez parcourir le tableau de l'indice 0 à N-1 avec une boucle POUR, évaluer 'Si T[i] == 5', et enregistrer ou afficher l'indice i à chaque fois que la condition est vérifiée.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments. Expliciter la démarche, le modèle retenu et la gestion des bornes.

Résolution pas à pas :
Étape 1 : Choix du modèle algorithmique. Le problème nécessite la vérification de chaque case du tableau sans interruption prématurée. Le modèle retenu est le 'Parcours complet avec traitement conditionné'. Étape 2 : Détermination des bornes d'indices. Convention d'indexation à 0 : pour un tableau de N éléments, la première case a l'indice 0 et la dernière case l'indice N - 1. Étape 3 : Sélection de la structure d'itération. Nombre d'itérations connu et fixe -> Boucle bornée 'Pour i allant de 0 à N - 1'. Étape 4 : Expression du filtre et du traitement. À chaque itération i, tester 'Si T[i] > 0 Alors', puis exécuter 'Afficher(T[i])'.

Résultat : Algorithme AffichagePositifs Données T : Tableau d'entiers N : Entier (taille du tableau) Variables i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire : L'algorithme garantit la visite exhaustive des N éléments. L'arrêt de la boucle à N-1 évite tout risque de dépassement d'indice (Out of Bounds).

■ Auto-Évaluation Flash d'Examen

Q : Quel est le risque critique si l'on écrit une boucle 'Pour i allant de 0 à N' sur un tableau de N éléments indicé à partir de 0 ?

R : Un risque de dépassement de capacité d'indice (Out of Bounds / Segmentation Fault). La case d'indice N n'existe pas dans un tableau de N éléments indicé de 0 à N-1.

Q : Combien de conditions d'arrêt minimales doit comporter la structure itérative d'une recherche de première occurrence ?

R : Deux conditions d'arrêt minimales : soit l'élément recherché a été trouvé (succès), soit la fin du tableau a été atteinte sans succès (i >= N).

Q : Pourquoi qualifie-t-on la démarche d'initialisation d'un algorithme de recherche de première occurrence de 'pessimiste' ?

R : Parce qu'on pose l'hypothèse de départ que l'élément recherché n'existe pas dans le tableau (ex: indicateur booléen 'trouve' initialisé à FAUX ou indice d'emplacement initialisé à -1) avant de commencer l'exploration.